

Reviewer Pack — Minimal Geometric Entropy of Binary Trees and Circuit Depth

Entry 81797e25eaa2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 09:43:00 UTC

Minimal Geometric Entropy of Binary Trees and Circuit Depth

Entry ID: 81797e25eaa2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 09:43:00 UTC

1. Conjecture

Field A (mathematical branch): Fractal Geometry **Field B** (complexity object): Boolean Circuits (Circuit Depth)

Statement:

The minimal geometric entropy of the binary tree representing a CNF formula is linearly correlated with its circuit depth, such that $H_{\min}(G) = \Theta(d(G))$, where G is the binary tree associated with the CNF and $d(G)$ is the depth of the minimum size circuit computing the function represented by G .

Rationale (proposer's reasoning):

Fractal geometry provides a natural framework to study the complexity of discrete structures due to its emphasis on self-similarity and dimensionality. The geometric entropy of a binary tree could capture inherent structural complexity, potentially offering insights into circuit complexity that are not evident from traditional measures like circuit size or depth.

Taxonomy category: Fractal_Geometry_x_Boolean_Circuits (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c0cbadd1c756fe9f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if a statistically significant linear correlation (p -value ≤ 0.05) exists between the computed geometric entropy of binary trees associated with CNF formulas and their corresponding circuit depths, with at least 80% of the data points lying within ± 3 standard deviations from the mean.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(num_vars, num_clauses):
        cnf = []
        for _ in range(num_clauses):
            clause = [random.randint(1, num_vars), -random.randint(1, num_vars)]
            cnf.append(clause)
        return cnf

    def binary_tree(cnf):
        if not cnf:
            return None
        root = (cnf[0][0], cnf[0][1])
        left_cnf = [c for c in cnf if c[0] == root[0]]
        right_cnf = [c for c in cnf if c[1] == root[1]]
        return (root, binary_tree(left_cnf), binary_tree(right_cnf))

    def geometric_entropy(tree):
        if not tree:
            return 0
        root, left, right = tree
        entropy = 1 + geometric_entropy(left) + geometric_entropy(right)
        return entropy

    def circuit_depth(cnf):
        if not cnf:
            return 0
        depth = 1
        for clause in cnf:
            depth = max(depth, max(abs(clause[0]), abs(clause[1])))
        return depth
```

```

n_max = 40
instances_tested = 0
metric_values = []

for n in range(5, n_max + 1):
    for _ in range(6): # Ensure at least 30 instances per seed
        cnf = generate_cnf(n, random.randint(2 * n, 4 * n))
        tree = binary_tree(cnf)
        entropy = geometric_entropy(tree)
        depth = circuit_depth(cnf)
        metric_values.append((entropy, depth))
        instances_tested += 1

if not metric_values:
    return {
        "metric_name": "geometric_entropy",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

entropy_values, depth_values = zip(*metric_values)
mean_entropy = sum(entropy_values) / len(entropy_values)
mean_depth = sum(depth_values) / len(depth_values)
std_dev = math.sqrt(sum((x - mean_depth) ** 2 for x in depth_values) / len(depth_values))

support_fraction = sum(abs(x - y) <= 3 * std_dev for x, y in zip(entropy_values, depth_values))

return {
    "metric_name": "geometric_entropy",
    "metric_value": mean_depth,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 7 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_v
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fra
    elif any(not r["conjecture_holds"] for r in results):

```

```

    first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d0c046ec.py", line 96, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d0c046ec.py", line 58, in run_trial
    tree = binary_tree(cnf)
           ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d0c046ec.py", line 34, in binary_tree
    return (root, binary_tree(left_cnf), binary_tree(right_cnf))
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d0c046ec.py", line 34, in binary_tree
    return (root, binary_tree(left_cnf), binary_tree(right_cnf))
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d0c046ec.py", line 34, in binary_tree
    return (root, binary_tree(left_cnf), binary_tree(right_cnf))
              ^^^^^^^^^^^^^^^^^^^^^^^^^
[Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Re-run the test with increased recursion limits and ensure that it completes without crashing. If the test passes, re-evaluate the conjecture based on the results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17852
2	preregistration	ollama_remote	glm4:latest	0	0	11893
3	novelty	ollama_remote	glm4:latest	0	0	8711
4	novelty	ollama_remote	glm4:latest	0	0	8936
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34982
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12086
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11352
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47065
9	judge	ollama_remote	glm4:latest	0	0	78442

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 231320 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/81797e25eaa2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/81797e25eaa2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/81797e25eaa2.tar.gz` (if generated)